Complex Numbers and AC Circuits

Engineering Computation with Python — Chapter 8

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1 Rectangular and Polar Form

2 Impedance

3 Summary

Table of Contents

1 Rectangular and Polar Form

2 Impedance

3 Summary

DC Needs Real Numbers. AC Needs One More Idea.

Your circuits course starts with **DC**: current flows one way, Ohm's law is simple, $V = IR$.

Real power grids, audio equipment, and wireless devices run on **AC**: voltage and current oscillate, and components shift their **timing (phase)** relative to each other.

The brilliant shortcut

Treat AC signals as complex numbers. The exact same simple algebra used for DC circuits then works for AC — no trigonometric identities required.

Python supports complex numbers natively. No extra library needed.

Two Equivalent Ways to Write a Complex Number

Rectangular form — real and imaginary parts separately:

$$Z = a + jb.$$

Polar form — magnitude and direction:

$$Z = |Z|\angle\theta, \qquad |Z| = \sqrt{a^2 + b^2}, \qquad \theta = \arctan\left(\frac{b}{a}\right).$$

Going back the other way uses trigonometry:

$$a = |Z|\cos\theta, \qquad b = |Z|\sin\theta.$$

Rectangular tells you resistance and reactance separately; polar tells you the overall opposition and the timing shift.

© Munim Zabidi

Converting Between Forms in Python

```python
import numpy as np

Z = 3 + 4j                      # rectangular form

# Rectangular -> polar
magnitude = abs(Z)
angle_deg = np.degrees(np.angle(Z))
print(f"|Z| = {magnitude:.2f}, angle = {angle_deg:.2f} degrees")

# Polar -> rectangular
a = magnitude * np.cos(np.radians(angle_deg))
b = magnitude * np.sin(np.radians(angle_deg))
print(f"back to rectangular: {a:.2f} + {b:.2f}j")
```

```
|Z| = 5.00, angle = 53.13 degrees
back to rectangular:  3.00 + 4.00j
```

Why 3–4–5, Not 3–4–7?

A resistance of 3 Ω and a reactance of 4 Ω combine to a total opposition of 5 Ω — not 7 Ω.

Why they don't simply add

Reactance and resistance act a quarter-cycle apart in time. That timing offset is precisely what the imaginary unit encodes — it is the familiar right-triangle relationship in disguise.

© Muním Zabidi

Table of Contents

1 Rectangular and Polar Form

2 Impedance

3 Summary

Impedance: The AC Generalization of Resistance

In DC circuits, the relevant quantity is **resistance** (R). In AC circuits, this expands to **impedance** (Z):

$$Z = R + jX.$$

Real part: R

Opposition to current that dissipates energy as heat. Independent of frequency.

Imaginary part: X (reactance)

Opposition from magnetic (inductor) or electric (capacitor) fields. Stores and releases energy — no dissipation, just a timing shift.

Mathematics uses $i = \sqrt{-1}$; electrical engineering uses j, because i is already current. Python follows the EE convention.

© Muním Zabidi

Reactance Depends on Frequency

At angular frequency $\omega = 2\pi f$:

Inductors (L)

Oppose rapid changes in current.

$$X_L = \omega L, \qquad +jX_L.$$

Reactance grows with frequency.

Capacitors (C)

Oppose rapid changes in voltage.

$$X_C = \frac{1}{\omega C}, \qquad -jX_C.$$

Reactance shrinks with frequency.

Writing Complex Numbers in Python

```
# Defining complex numbers directly
Z1 = 40 + 30j     # 40 ohms real, +30 ohms reactance
Z2 = 20 - 15j     # 20 ohms real, -15 ohms reactance

print(type(Z1))   # Output: <class 'complex'>
```

Watch out

A number must come immediately before `j` (`3j`, `1j`). Writing just `j` alone is an ordinary variable name — a `NameError` if undefined.

Every Property, One Call Away

```python
import numpy as np

Z = 40 + 22.35j

print("Real part (R):", Z.real)          # 40.0
print("Imaginary part (X):", Z.imag)     # 22.35

magnitude = abs(Z)
print(f"Total opposition: {magnitude:.2f} Ohms")  # 45.82

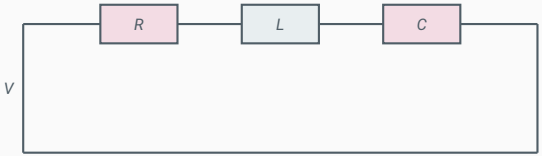
phase_degrees = np.degrees(np.angle(Z))
print(f"Phase shift: {phase_degrees:.1f} deg")     # 29.2
```

$R = 40\,\Omega$, $L = 200\,\text{mH}$, $C = 50\,\mu\text{F}$, $f = 60\,\text{Hz}$.

Series rule

The same current flows through every component, so impedances add:

$$Z = Z_R + Z_L + Z_C.$$



© Muním Zabidi

Solving the Series Circuit in Python

```python
import numpy as np

R, L, C, f = 40.0, 0.2, 50e-6, 60.0
omega = 2 * np.pi * f

Z_R = R
Z_L = 1j * omega * L
Z_C = -1j / (omega * C)

Z_total = Z_R + Z_L + Z_C   # series: impedances just add!

print(f"Total Impedance:   {Z_total.real:.1f} + {Z_total.imag:.2f}j Ohms")
print(f"Magnitude:         {np.abs(Z_total):.2f} Ohms")
print(f"Phase Angle:       {np.degrees(np.angle(Z_total)):.1f} degrees")
```

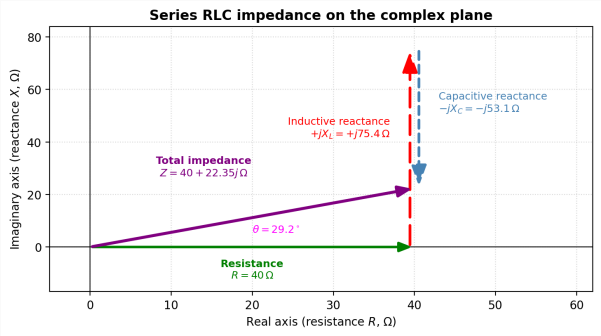
Reading the Series Result

```
Total Impedance:  40.0 + 22.35j Ohms
Magnitude:  45.82 Ohms    Phase Angle:  29.2 degrees
```

Interpret

Without a single trigonometry problem: the circuit opposes current with magnitude 45.82 Ω, and current lags voltage by 29.2° — a positive angle means inductive reactance dominates at this frequency.

Visualizing Impedance on the Complex Plane



Start at the origin: step 40 Ω right (R), step up 75.4 Ω ($+jX_L$), step down 53.1 Ω ($-jX_C$).

The purple vector from origin to $40 + 22.35j$ is the total impedance Z: its length is the magnitude, its angle is the phase.

Inductors and Capacitors Cancel Each Other

The vector sum shows visually how inductors and capacitors actively oppose each other's effects.

Reactances can cancel

If the frequency were tuned so that $X_L = X_C$, the upward and downward reactance steps would be exactly equal, and the total impedance would lie flat on the real axis — a result your circuits course will study further.

Table of Contents

1 Rectangular and Polar Form

2 Impedance

3 Summary

What This Chapter Covered

- Python writes complex numbers with a `j` suffix (`3 + 4j`); must be `1j`, never bare `j`.
- **Rectangular** suits $+/-$; **polar** suits $\times/\div$. `abs()` and `np.angle()` convert between them.
- **Impedance** $Z = R + jX$ generalises resistance; $X_L = \omega L$ (positive), $X_C = -1/\omega C$ (negative).
- Series impedances add; parallel impedances combine by reciprocal — same rule as DC resistors, complex arithmetic.
- **Magnitude** sets current flow; **phase** says whether the load is capacitive or inductive.
- Parallel circuits, AC power, and resonance all follow the same pattern of complex-number reasoning introduced here — your circuits course will build further on this foundation.

© Munim Zabidi

This chapter turned AC circuit analysis into ordinary algebra with complex numbers. The next chapter extends this to circuits with multiple loops and nodes.

- Chapter 9: Matrices and Circuit Equations — representing a whole circuit as $A\mathbf{x} = \mathbf{b}$ and solving it in one line.
- The complex-number skills here carry over directly: AC mesh equations are solved with the same `numpy` linear algebra, just with complex entries.

Next: Matrices and Circuit Equations

© Muním Zabidi